

Factory method pattern

In object-oriented programming, the **factory method pattern** is a [design pattern](#) that uses factory methods to deal with the problem of [creating objects](#) without having to specify their exact [classes](#). Rather than by calling a [constructor](#), this is accomplished by invoking a factory method to create an object. Factory methods can be specified in an [interface](#) and implemented by subclasses or implemented in a base class and optionally [overridden](#) by subclasses. It is one of the 23 classic design patterns described in the book *Design Patterns* and is subcategorized as a [creational pattern](#).^[1]

Overview

The factory method design pattern solves problems such as:

- How can an object's [subclasses](#) redefine its subsequent and distinct implementation? The pattern involves creation of a factory method within the [superclass](#) that defers the object's creation to a subclass's factory method.
- How can an object's instantiation be deferred to a subclass? Create an object by calling a factory method instead of directly calling a constructor.

This enables the creation of subclasses that can change the way in which an object is created (for example, by redefining which class to instantiate).

Definition

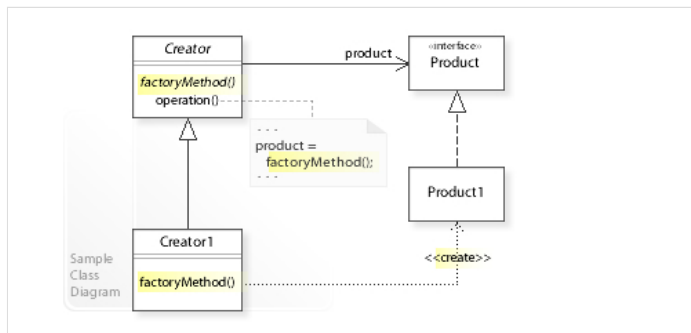
According to *Design Patterns: Elements of Reusable Object-Oriented Software*: "Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory method lets a class defer instantiation to subclasses."^[2]

Creating an object often requires complex processes not appropriate to include within a composing object. The object's creation may lead to a significant duplication of code, may require information inaccessible to the composing object, may not provide a sufficient level of abstraction or may otherwise not be included in the composing object's [concerns](#). The factory method design pattern handles these problems by defining a separate [method](#) for creating the objects, which subclasses can then override to specify the [derived type](#) of product that will be created.

The factory method pattern relies on inheritance, as object creation is delegated to subclasses that implement the factory method to create objects.^[3] The pattern can also rely on the implementation of an [interface](#).

Structure

UML class diagram



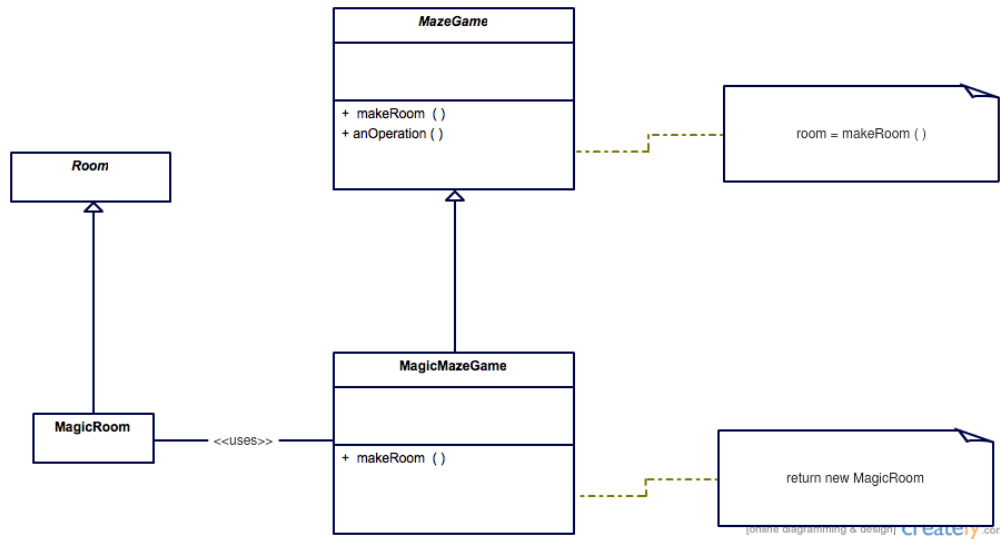
A sample UML class diagram for the Factory Method design pattern.^[4]

In the above [UML class diagram](#), the **Creator** class that requires a **Product** object does not instantiate the **Product1** class directly. Instead, the **Creator** refers to a separate **factoryMethod()** to create a product object, which makes the **Creator** independent of the exact concrete class that is instantiated. Subclasses of **Creator** can redefine which class to instantiate. In this example, the **Creator1** subclass implements the abstract **factoryMethod()** by instantiating the **Product1** class.

Examples

Structure

A maze game may be played in two modes, one with regular rooms that are only connected with adjacent rooms, and one with magic rooms that allow players to be transported at random.



Room is the base class for a final product (MagicRoom or OrdinaryRoom). MazeGame declares the abstract factory method to produce such a base product. MagicRoom and OrdinaryRoom are subclasses of the base product implementing the final product. MagicMazeGame and OrdinaryMazeGame are subclasses of MazeGame implementing the factory method producing the final products. Factory methods thus decouple callers (MazeGame) from the implementation of the concrete classes. This makes the new operator redundant, allows adherence to the open-closed principle and makes the final product more flexible in the event of change.

Example implementations

C++

This C++23 implementation is based on the pre C++98 implementation in the *Design Patterns* book.^[5]

```

import std;
using std::unique_ptr;

// defines the interface of objects the factory method creates.
class Product {
public:
    enum class Id: char {
        MINE,
        YOURS,
    };

    virtual void print() = 0;
    virtual ~Product() = default;
};

// implements the Product interface.
class ConcreteProductMine: public Product {
public:
    void print() {
        std::println("this={} print MINE", this);
    }
};

// implements the Product interface.
class ConcreteProductYours: public Product {
public:
    void print() {
        std::println("this={} print YOURS", this);
    }
};

// declares the factory method, which returns an object of type Product.
class Creator {
public:
    virtual unique_ptr<Product> create(Product::Id id) {
        switch (id) {
            case Product::Id::MINE:
                return std::make_unique<ConcreteProductMine>();
            case Product::Id::YOURS:
                return std::make_unique<ConcreteProductYours>();
            // repeat for remaining products
        }
        std::unreachable();
    }

    virtual ~Creator() = default;
};

int main(int argc, char* argv[]) {
    unique_ptr<Creator> creator = std::make_unique<Creator>();
    unique_ptr<Product> product = creator->create(Product::Id::MINE);
    product->print();

    product = creator->create(Product::Id::YOURS);
    product->print();
}
  
```

The program output is like

```
this=0x6e5e90 print MINE
this=0x6e62c0 print YOURS
```

C#

```
// Empty vocabulary of actual object
public interface IPerson
{
    string GetName();
}

public class Villager : IPerson
{
    public string GetName()
    {
        return "Village Person";
    }
}

public class CityPerson : IPerson
{
    public string GetName()
    {
        return "City Person";
    }
}

public enum PersonType
{
    Rural,
    Urban
}

/// <summary>
/// Implementation of Factory - Used to create objects.
/// </summary>
public class PersonFactory
{
    public IPerson GetPerson(PersonType type)
    {
        switch (type)
        {
            case PersonType.Rural:
                return new Villager();
            case PersonType.Urban:
                return new CityPerson();
            default:
                throw new NotSupportedException();
        }
    }
}
```

The above code depicts the creation of an interface called `IPerson` and two implementations called `Villager` and `CityPerson`. Based on the type passed to the `PersonFactory` object, the original concrete object is returned as the interface `IPerson`.

A factory method is just an addition to the `PersonFactory` class. It creates the object of the class through interfaces but also allows the subclass to decide which class is instantiated.

```
public interface IProduct
{
    string GetName();
    bool SetPrice(double price);
}

public class Phone : IProduct
{
    private double _price;

    public string GetName()
    {
        return "Apple TouchPad";
    }

    public bool SetPrice(double price)
    {
        _price = price;
        return true;
    }
}

// Almost same as Factory, just an additional exposure to do something with the created method
public abstract class ProductAbstractFactory
{
    protected abstract IProduct MakeProduct();

    public IProduct GetObject() // Implementation of Factory Method.
    {
        return this.MakeProduct();
    }
}

public class PhoneConcreteFactory : ProductAbstractFactory
{
    protected override IProduct MakeProduct()
    {
        IProduct product = new Phone();
        // Do something with the object after receiving it
    }
}
```

```

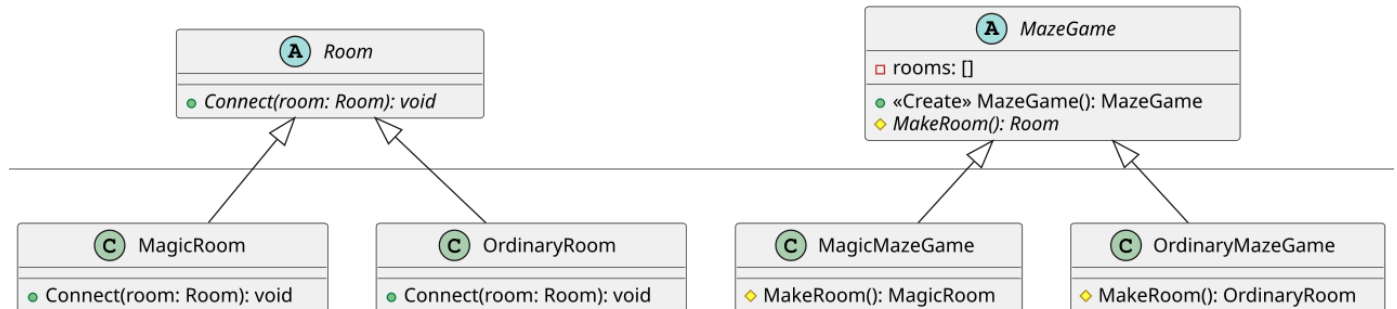
        product.SetPrice(20.30);
        return product;
    }
}

```

In this example, `MakeProduct` is used in `concreteFactory`. As a result, `MakeProduct()` may be invoked in order to retrieve it from the `IProduct`. Custom logic could run after the object is obtained in the concrete factory method. `GetObject` is made abstract in the factory interface.

Java

This [Java](#) example is similar to one in the book *Design Patterns*.



The `MazeGame` uses `Room` but delegates the responsibility of creating `Room` objects to its subclasses that create the concrete classes. The regular game mode could use this template method:

```

import java.util.ArrayList;
import java.util.List;

abstract class Room {
    abstract void connect(Room room);
}

class MagicRoom extends Room {
    public void connect(Room room) {}
}

class OrdinaryRoom extends Room {
    public void connect(Room room) {}
}

abstract class MazeGame {
    private final List<Room> rooms = new ArrayList<>();

    public MazeGame() {
        Room room1 = makeRoom();
        Room room2 = makeRoom();
        room1.connect(room2);
        rooms.add(room1);
        rooms.add(room2);
    }

    abstract protected Room makeRoom();
}

```

The `MazeGame` constructor is a template method that adds some common logic. It refers to the `makeRoom()` factory method that encapsulates the creation of rooms such that other rooms can be used in a subclass. To implement the other game mode that has magic rooms, the `makeRoom` method may be overridden:

```

class MagicMazeGame extends MazeGame {
    @Override
    protected MagicRoom makeRoom() {
        return new MagicRoom();
    }
}

class OrdinaryMazeGame extends MazeGame {
    @Override
    protected OrdinaryRoom makeRoom() {
        return new OrdinaryRoom();
    }
}

MazeGame ordinaryGame = new OrdinaryMazeGame();
MazeGame magicGame = new MagicMazeGame();

```

PHP

This PHP example shows interface implementations instead of subclassing (however, the same can be achieved through subclassing). The factory method can also be defined as `public` and called directly by the client code (in contrast to the previous Java example).

```

/* Factory and car interfaces */

interface CarFactory
{
    public function makeCar(): Car;
}

```

```

}

interface Car
{
    public function getType(): string;
}

/* Concrete implementations of the factory and car */

class SedanFactory implements CarFactory
{
    public function makeCar(): Car
    {
        return new Sedan();
    }
}

class Sedan implements Car
{
    public function getType(): string
    {
        return 'Sedan';
    }
}

/* Client */

$factory = new SedanFactory();
$car = $factory->makeCar();
print $car->getType();

```

Python

This Python example employs the same as did the previous Java example.

```

from abc import ABC, abstractmethod

class MazeGame(ABC):
    def __init__(self) -> None:
        self.rooms = []
        self._prepare_rooms()

    def _prepare_rooms(self) -> None:
        room1 = self.make_room()
        room2 = self.make_room()

        room1.connect(room2)
        self.rooms.append(room1)
        self.rooms.append(room2)

    def play(self) -> None:
        print(f"Playing using {self.rooms[0]}")

    @abstractmethod
    def make_room(self):
        raise NotImplementedError("You should implement this!")

class MagicMazeGame(MazeGame):
    def make_room(self) -> "MagicRoom":
        return MagicRoom()

class OrdinaryMazeGame(MazeGame):
    def make_room(self) -> "OrdinaryRoom":
        return OrdinaryRoom()

class Room(ABC):
    def __init__(self) -> None:
        self.connected_rooms = []

    def connect(self, room: "Room") -> None:
        self.connected_rooms.append(room)

class MagicRoom(Room):
    def __str__(self) -> str:
        return "Magic room"

class OrdinaryRoom(Room):
    def __str__(self) -> str:
        return "Ordinary room"

ordinaryGame = OrdinaryMazeGame()
ordinaryGame.play()

magicGame = MagicMazeGame()
magicGame.play()

```

Uses

- In ADO.NET, IDbCommand.CreateCommand (<https://docs.microsoft.com/en-us/dotnet/api/system.data.idbcommand.createparameter?view=netframework-4.8>) is an example of the use of factory method to connect parallel class hierarchies.
- In Qt, QMainWindow::createPopupMenu (<http://qt-project.org/doc/qt-5.0/qtwidgets/qmainwindow.html#createPopupMenu>) Archived (<http://s://web.archive.org/web/20150719014739/http://qt-project.org/doc/qt-5.0/qtwidgets/qmainwindow.html#createPopupMenu>) 2015-07-19 at

the [Wayback Machine](#) is a factory method declared in a framework that can be overridden in application code.

- In Java, several factories are used in the `javax.xml.parsers` (<http://download.oracle.com/javase/1.5.0/docs/api/javax/xml/parsers/package-summary.html>) package, such as `javax.xml.parsers.DocumentBuilderFactory` or `javax.xml.parsers.SAXParserFactory`.
- In the HTML5 DOM API, the Document interface contains a `createElement()` factory method for creating specific elements of the `HTMLElement` interface.

See also

- *Design Patterns*, the highly influential book
- *Design pattern*, overview of design patterns in general
- Abstract factory pattern, a pattern often implemented using factory methods
- Builder pattern, another creational pattern
- Template method pattern, which may call factory methods
- Joshua Bloch's idea of a *static factory method* for which Bloch claims there is no direct equivalent in *Design Patterns*.

Notes

1. Gamma et al. 1995, p. 107.
2. Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN 0-201-63361-2.
3. Freeman, Eric; Robson, Elisabeth; Sierra, Kathy; Bates, Bert (2004). Hendrickson, Mike; Loukides, Mike (eds.). *Head First Design Patterns: A Brain-Friendly Guide* (<http://shop.oreilly.com/product/9780596007126.do>) (paperback). Vol. 1 (1st ed.). O'Reilly Media. p. 162. ISBN 978-0-596-00712-6. Retrieved 2012-09-12.
4. "The Factory Method design pattern - Structure and Collaboration" (<http://w3sdesign.com/?gr=c03&ugr=struct>). *w3sDesign.com*. Retrieved 2017-08-12.
5. Gamma et al. 1995, p. 122.

References

- Martin Fowler; Kent Beck; John Brant; William Opdyke; Don Roberts (June 1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley. ISBN 0-201-48567-2.
- Cox, Brad J. (1986). *Object-oriented programming: an evolutionary approach*. Addison-Wesley. ISBN 978-0-201-10393-9.
- Cohen, Tal; Gil, Joseph (2007). "Better Construction with Factories" (<http://tal.forum2.org/static/cv/Factories.pdf>) (PDF). *Journal of Object Technology*. **6** (6). Bertrand Meyer: 103. doi:10.5381/jot.2007.6.6.a3 (<https://doi.org/10.5381%2Fjot.2007.6.6.a3>). Retrieved 2007-03-12.

External links

- Factory Design Pattern (<http://designpattern.co.il/Factory.html>) Archived (<https://web.archive.org/web/20180110103637/http://designpattern.co.il/Factory.html>) 2018-01-10 at the Wayback Machine Implementation in Java
- Factory method in UML and in LePUS3 (<https://web.archive.org/web/20080503082912/http://www.lepus.org.uk/ref/companion/FactoryMethod.xml>) (a Design Description Language)
- Consider static factory methods (<http://drdobbs.com/java/208403883>) by Joshua Bloch

Retrieved from "https://en.wikipedia.org/w/index.php?title=Factory_method_pattern&oldid=1362959768"